

Stratégie 5 — Métriques et optimisation : lire, se méfier, ne pas se mentir

Pas de nouveau backtest LEAN dans cette leçon : on exploite les **six déjà mesurés** (Buy & Hold, SMA, RSI, Bollinger, MACD, RSI+risque). Deux compétences qui n'en font qu'une : **lire un rapport comme un pro** (et repérer les métriques qui mentent), puis **optimiser sans se mentir** (et reconnaître un backtest trop beau pour être vrai). La démonstration finale est brutale : le « meilleur » paramètre trouvé par optimisation s'effondre hors échantillon, battu par le réglage bête qu'on aurait détesté.

Les métriques : trois familles, plus une

Un rapport de backtest crache des dizaines de chiffres. Ils se rangent en trois familles — le **rendement** (Net Profit, CAR annualisé : combien on a gagné, rien sur le chemin), le **risque** (drawdown : la pire baisse depuis un plus-haut — ce qui fait *abandonner* une stratégie), et le **rendement ajusté au risque** : **Sharpe** (rendement ÷ volatilité totale), **Sortino** (ne pénalise que la volatilité à la baisse), **Calmar** (CAR ÷ drawdown max). C'est sur la troisième qu'on compare.

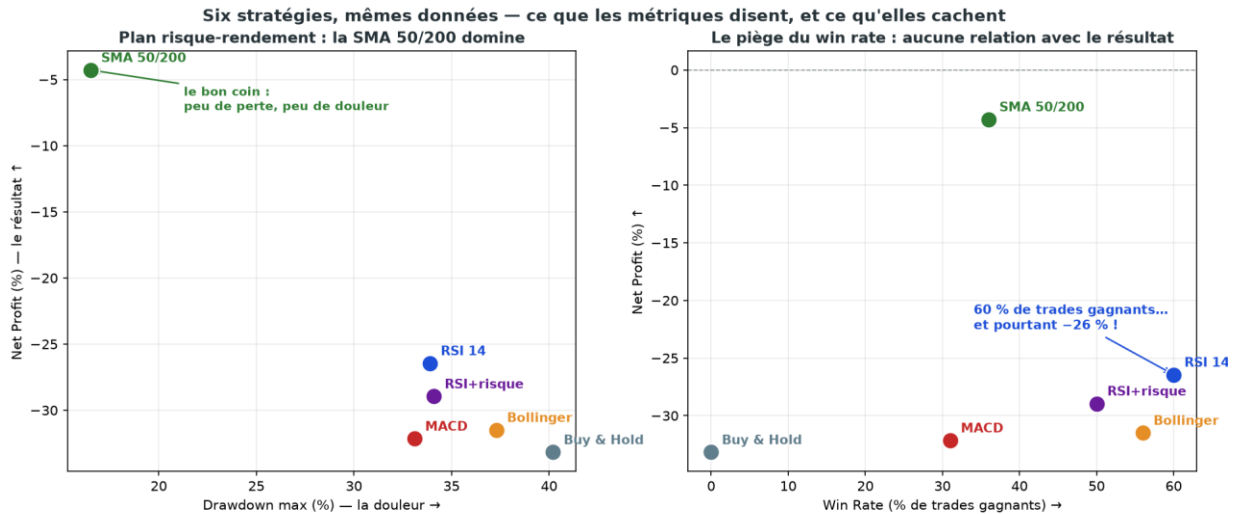
À part, la famille **par trade** : Win Rate (% de gagnants), Profit-Loss Ratio (gain moyen ÷ perte moyenne), et l'**expectancy** = $\text{win_rate} \times \text{gain_moyen} + \text{loss_rate} \times \text{perte_moyenne}$ — l'espérance de gain par trade, la seule à combiner les deux autres. Attention à l'unité : LEAN publie la sienne sous une autre forme — $\text{win_rate} \times \text{ratio_gain_perte} - \text{loss_rate}$, en **multiples de la perte moyenne** (sans unité) : c'est la colonne « Expectancy (LEAN) » du tableau. Même verdict, échelle différente.

Le script `fig_metriques.py` lit les `*-summary.json` des six runs et imprime le tableau :

```
python ../../Formations/Backtesting/LEAN/scripts/fig_metriques.py
```

Stratégie	Net Profit	Drawdown	Sharpe	Sortino	Win Rate	Gain/Perte	Expectancy (LEAN)	Frais
Buy & Hold	-33,2 %	40,2 %	-1,10	-1,51	0 %	—	0	40 \$
SMA 50/200	-4,3 %	16,5 %	-0,46	-0,57	36 %	1,39	-0,13	827 \$
RSI 14	-26,4 %	33,9 %	-0,99	-1,07	60 %	0,44	-0,14	1 030 \$
Bollinger	-31,5 %	37,3 %	-1,65	-1,33	56 %	0,42	-0,21	4 964 \$
MACD	-32,1 %	33,1 %	-1,64	-2,97	31 %	1,59	-0,21	11 692 \$
RSI+risque	-29,0 %	34,1 %	-1,30	-1,31	50 %	0,60	-0,20	1 321 \$

Deux anomalies sautent aux yeux : le RSI a le **meilleur win rate** (60 %) et perd gros ; le MACD a le **meilleur ratio gain/perte** (1,59) et perd aussi. On y vient.



Deux panneaux : à gauche le plan Net Profit contre drawdown maximal — la SMA 50/200 est seule dans le bon coin (haut-gauche), toutes les autres regroupées en bas à droite ; à droite Net Profit contre win rate — aucune relation, le RSI perd -26 % avec 60 % de trades gagnants.

Figure — Gauche : la SMA domine le plan risque-rendement. Droite : le win rate n'a **aucune** relation avec le résultat. (Script : `fig_metriques.py`)

Le plan risque-rendement (panneau gauche) situe tout d'un coup : le bon coin est en haut-gauche, et la SMA y est seule. Sharpe, Sortino et Calmar confirment — la SMA a les moins mauvais. **Les moins mauvais** : tous les Sharpe sont **négatifs**. Sur ce marché baissier, aucune stratégie longue ne « bat le fait de dormir » ; la SMA n'est pas bonne, elle est la *moins pire* — parce qu'elle reste à plat pendant les pires baisses.

Le piège du win rate

Le RSI gagne 60 % de ses trades — un débutant achèterait sur ce seul chiffre. Or :

RSI 14 : gain moyen = +3,6 % perte moyenne = -8,2 % (ratio gain/perte = 0,44)
 Expectancy = $0,60 \times 3,6 \% + 0,40 \times (-8,2 \%) = +2,16 \% - 3,28 \% = -1,1 \% \text{ par trade}$
 → **NÉGATIF**

(En unités LEAN : $0,60 \times 0,44 - 0,40 = -0,14$ — le chiffre du tableau. Autre échelle, même verdict.)

Gagner **souvent** un peu et perdre **rarement** beaucoup = **perdre**. Le MACD illustre le piège symétrique : ses gagnants dépassent ses perdants (1,59) mais il ne gagne que 31 % du temps — expectancy négative aussi. **Ni le win rate seul, ni le ratio seul** : seule leur combinaison — l'expectancy — dit la vérité. C'est d'ailleurs le levier des vendeurs de systèmes : resserrer la sortie gonfle le win rate (« 80 % de réussite ! ») en rabotant le gain moyen — l'expectancy peut empirer pendant que la vitrine s'améliore. Un rapport honnête publie toujours win rate **et** ratio gain/perte ensemble (ou directement l'expectancy), **nets de frais**.

Et même bien lues, ces métriques restent fragiles pour une raison plus profonde : elles sortent d'**un seul échantillon** (six mois, un marché, un régime baissier — le Probabilistic Sharpe Ratio de LEAN, proche de 0 % ici, quantifie cette indétermination), le **régime domine tout** (en marché haussier le classement s'inverserait probablement), et elles sont **malléables** (période d'annualisation, trade « exceptionnel » exclu, Sharpe plutôt que Sortino — chaque choix flatte ou punit). Une métrique **résume**, elle ne **prouve** pas : elle compare des stratégies sur les mêmes données, elle ne certifie jamais qu'une stratégie « marche ». Pour ça, il faut la tester **hors de l'échantillon qui l'a vue naître**.

L'optimisation : mémoriser le passé

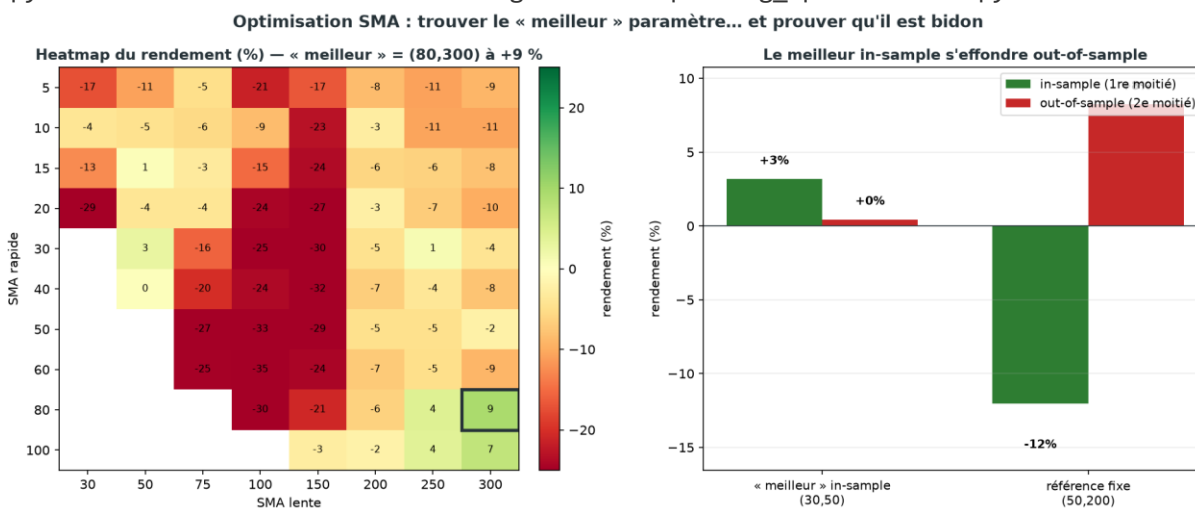
Jusqu'ici on a choisi les paramètres (50/200, 30/70...) sans les justifier. La tentation : chercher **les meilleurs** — balayer toutes les combinaisons, garder celle qui rapporte le plus. Le problème : sur des données passées, il y a **toujours** une combinaison qui brille, **par hasard**. Avec assez d'essais, on finit par trouver le réglage qui colle aux accidents précis de l'échantillon. On ne capture pas une régularité du marché : on **mémorise le bruit**. C'est le **sur-apprentissage** (*overfitting*).

Relancer LEAN des centaines de fois serait interminable. Pour comparer des paramètres *entre eux*, un backtest **vectorisé** suffit — croisement SMA long/flat, fill au close, frais 0,04 %, dans `fig_optimisation.py` :

```
def backtest(nf, nl, i0=0, i1=None, fee=0.0004):
    rf, rl = SMA[nf], SMA[nl]
    eq, pos = 1.0, 0
    for i in range(i0 + 1, i1):
        if pos:
            eq *= close[i] / close[i - 1]          # on subit le rendement de la
barre si investi                                # long si rapide > lente, flat
            state = 1 if rf[i] > rl[i] else 0        # long si rapide > lente, flat
        sinon
            if state != pos:
                eq *= (1 - fee); pos = state          # frais à chaque changement
    d'état
    return eq - 1.0
```

Il tombe à ~0,5 point de LEAN sur (50,200) — écart constant (tampon 99,7 %, frais sur le notionnel), sans effet sur un classement **relatif**. C'est exactement le genre d'écart que la section 03 (parité) expliquera au centime.

python ../../Formations/Backtesting/LEAN/scripts/fig_optimisation.py



Deux panneaux : à gauche une heatmap du rendement pour chaque combinaison SMA rapide × SMA lente, presque entièrement rouge avec un petit coin vert vers (80,300) à +9 % ; à droite un histogramme in-sample vs out-of-sample — le meilleur in-sample (30,50) passe de +3,2 % à +0,4 %, la référence fixe (50,200) passe de -12 % à +8,2 %.

Figure — Gauche : sur toute la période, un seul coin est vert — tentant. Droite : le démasquage. (Script : `fig_optimisation.py`)

La heatmap (gauche) montre l'espace des paramètres : l'immense majorité **perd**, une poignée de réglages « gagne » dans un coin — le meilleur, (80,300), affiche **+9 %**. Un optimiseur naïf s'arrêterait là. C'est un piège : ce +9 % décrit **le passé**, pas une règle.

Le démasquage in-sample / out-of-sample

Le seul test honnête : couper l'historique en deux, optimiser sur la **première** moitié (in-sample), tester le gagnant sur la **seconde** (out-of-sample), jamais vue :

Réglage	in-sample (1 ^{re} moitié)	out-of-sample (2 ^e moitié)
« meilleur » in-sample = (30,50)	+3,2 %	+0,4 %
référence fixe (50,200)	-12,0 %	+8,2 %

Lis ce tableau lentement, c'est toute la leçon. Sur la 1^{re} moitié, l'optimiseur choisit (30,50) : +3,2 %. Sur le « futur », ce champion tombe à +0,4 % — il avait mémorisé les accidents de la 1^{re} moitié. Pire : la référence fixe (50,200), **pas** optimisée, qui faisait un catastrophique -12 % in-sample, réalise **+8,2 % out-of-sample** — vingt fois mieux que l'« optimisée ». **Optimiser a activement nuit**. C'est la signature du sur-apprentissage : plus on colle à l'in-sample, moins on tient en out-of-sample. Les bons paramètres ne sont pas les plus *performants* sur un historique, mais les plus **robustes** à travers des marchés différents.

Les défenses du pro

- **Split IS/OOS systématique** : réserver l'OOS **avant** de commencer, ne jamais juger un réglage sur les données qui l'ont choisi — et ne dépenser l'OOS **qu'une fois** (dès qu'on le regarde pour réajuster, il devient de l'IS).
- **Walk-forward** : ré-optimiser sur fenêtre glissante, tester toujours sur la fenêtre suivante. Si le réglage optimal change tout le temps, il n'y a pas de régularité à capturer.
- **Préférer les plateaux aux pics** : une **zone** large de réglages corrects vaut mieux qu'un pic isolé entouré de rouge — le pic est fragile.
- **Compter les essais** : plus on teste de combinaisons, plus un « gagnant » par chance est probable (le *deflated Sharpe* de Bailey & López de Prado corrige de ça). Avec ~50 combinaisons sur un marché volatil, trouver un +9 % par pur hasard est très probable — le +9 % de (80,300) ne prouve **rien**.

C'est précisément pour ça que la sous-section 02 (vectorbt) existe : balayer des milliers de combinaisons en millisecondes et outiller le walk-forward proprement. Mais la règle ne change pas : **le juge, c'est l'out-of-sample**. Et le sur-apprentissage frappe *tous* les paramètres — périodes d'indicateurs comme seuils de stop de la Stratégie 4.

Ce qu'il faut retenir

- **Trois familles + une** : rendement, risque, ajusté au risque (on compare sur celle-là), et par-trade — où seule l'**expectancy** (win rate × gains combinés) dit la vérité. 60 % de réussite et -26 % coexistent sans contradiction.
- **Tous les Sharpe sont négatifs ici** : la SMA est la *moins pire*, pas bonne. Une métrique résume, compare — mais ne certifie jamais : échantillon unique, régime dominant, chiffres malléables.
- **Le meilleur in-sample s'effondre out-of-sample** ((30,50) : +3,2 % → +0,4 %), battu par la référence fixe (+8,2 % OOS). Optimiser sans OOS, c'est mémoriser le bruit.
- **Défenses** : split IS/OOS réservé d'avance (dépensé une seule fois), walk-forward, plateaux plutôt que pics, et compter ses essais.

Précédent : [Stratégie 4 — La gestion du risque](#) · Suite : [Stratégie 6 — La stratégie avancée](#) : tout assembler — régime, momentum, sizing au risque, stops — dans un vrai backtest final.